

SymHome - Architecture et explication du code

Objectif du projet

SymHome est une application e-commerce Symfony pour vendre des meubles. Le client consulte le catalogue, ajoute des meubles dans son panier, valide une commande, puis consulte son historique. L'administrateur gère les meubles, catégories, commandes et utilisateurs depuis l'espace admin.

Comptes de demonstration

Type	Email	Mot de passe
Utilisateur	ahmed@example.com	password123
Utilisateur	fatma@example.com	password123
Utilisateur	aziz@example.com	password123
Admin	rayenessid15@gmail.com	essid
Admin	adem.essid@example.com	essid
Admin	ameni.wesleti@example.com	wesleti

Les utilisateurs standards servent aux tests du panier, du checkout et de l'historique.

Les administrateurs ont ROLE_ADMIN. Comme User::getRoles() ajoute toujours ROLE_USER, un administrateur conserve aussi les droits utilisateur.

Architecture generale

Le projet suit l'architecture MVC de Symfony.

Couche	Dossiers principaux	Rôle
Contrôleurs	src/Controller	Reçoivent les requêtes HTTP, appellent les repositories et rendent les templates.
Entités	src/Entity	Représentent les tables Doctrine: User, Catégorie, Meuble, Commande, LigneCommande.
Formulaires	src/Form	Décrivent les champs et validations des formulaires Symfony.
Repositories	src/Repository	Centralisent les requêtes Doctrine vers la base.
Templates	templates	Affichent les pages Twig côté utilisateur et admin.
Fixtures	src/DataFixtures/AppFixtures.php	Créent les données de démonstration.
Configuration	config/packages	Configure sécurité, Doctrine, Twig, mailer, reset password.

Entites principales

User stocke les comptes, rôles, mot de passe, nom, prénom et vérification email. La relation OneToMany vers Commande permet de retrouver toutes les commandes d'un utilisateur.

Catégorie regroupe les meubles par famille, par exemple séjour, chambre, bureau ou cuisine.

Meuble représente un produit: nom, description, prix, stock, image et catégorie. Il est relié aux lignes de commande.

Commande représente une commande complète: numéro unique, statut, total, date, adresse de livraison, utilisateur et liste des lignes.

LigneCommande représente un article dans une commande. Elle relie une commande à un meuble avec une quantité et un prix unitaire. Sa méthode getSousTotal() calcule $\text{prixUnitaire} \times \text{quantite}$.

Flux panier et commande

1. Le panier est stocke en session sous forme meubleId => quantite.
2. CommandeController::checkout() relit les meubles depuis la base pour construire le panier complet.
3. A la validation, une nouvelle Commande est creee et associee a l'utilisateur connecte.
4. Pour chaque article du panier, une LigneCommande est creee et ajoutee a la commande.
5. Commande::calculerTotal() additionne les sous-totaux.
6. La commande est persistee, le stock est decremente, puis le panier est vide.
7. L'utilisateur peut consulter la confirmation et l'historique de ses commandes.

Securite

La configuration config/packages/security.yaml utilise l'entite User comme provider. Les mots de passe sont en clair pour ce projet de demonstration (algorithm: plaintext). En production, il faut utiliser un hasher robuste comme auto.

Regles d'accès:

```
URL  Role requis
/admin  ROLE_ADMIN
/commande  ROLE_USER
```

Espace admin

Les controleurs dans src/Controller/Admin sont proteges par #[IsGranted('ROLE_ADMIN')]. Ils gerent le dashboard, les meubles, les categories, les commandes et les utilisateurs. UserCrudController::toggleAdmin() permet de promouvoir ou retirer le role administrateur.

Donnees de test

AppFixtures.php cree:

- 3 utilisateurs standards: Ahmed, Fatma, Aziz.
- 3 administrateurs: Rayen Essid, Adem Essid et Ameni Wesleti.
- Des categories et meubles de demonstration.
- 20 commandes de demonstration, liees uniquement aux utilisateurs standards.

Commandes CLI de verification

Commandes utiles pour verifier le projet:

```
php bin/console lint:yml config
php bin/console lint:twig templates
php bin/console debug:router
php bin/console doctrine:schema:validate --skip-sync
php bin/phpunit
```

Verification effectuee le 18/05/2026:

```
Commande  Resultat
php -l src/DataFixtures/AppFixtures.php  OK, aucune erreur de syntaxe
php bin/console lint:yml config  OK, 28 fichiers YAML valides
```

```
php bin/console lint:twig templates OK, 22 fichiers Twig valides
php bin/console debug:router OK, routes /commande presentes
php bin/console doctrine:schema:validate --skip-sync OK, mapping Doctrine correct
php bin/console doctrine:fixtures:load --env=test OK, fixtures chargees sur SQLite temporaire
php bin/phpunit OK, 4 tests et 4 assertions
```

Pour recharger les donnees de demonstration:

```
php bin/console doctrine:fixtures:load
```

Cette commande supprime puis recree les donnees de la base configuree dans .env.